

CVE Data Warehouse

OLAP analysis of software vulnerabilities, 2020–2026

Alessia Angelé – 2003000

Data Management · Sapienza University of Rome

Contents

- 1 Project idea & questions
- 2 Data sources
- 3 ETL pipeline
- 4 Dimensional Fact Models

- 5 Logical schema
- 6 OLAP queries & results
- 7 Conclusions

Project idea

CVSS says how bad a vulnerability could be. It does not say whether anyone will exploit it.



QUESTIONS THE WAREHOUSE ANSWERS

How fast is the number of vulnerabilities growing, and is their severity changing?	Q1
Which weaknesses and attack classes get exploited, relative to how common they are?	Q2a · Q2b · Q3
Does CVSS severity agree with predicted exploitation (EPSS)?	Q5
Which vendors are most at risk, and how fast are their flaws exploited?	Q4 · Q6 · Q7

Two business processes → two fact tables

a CVE is published in NVD · a CVE is added to CISA KEV

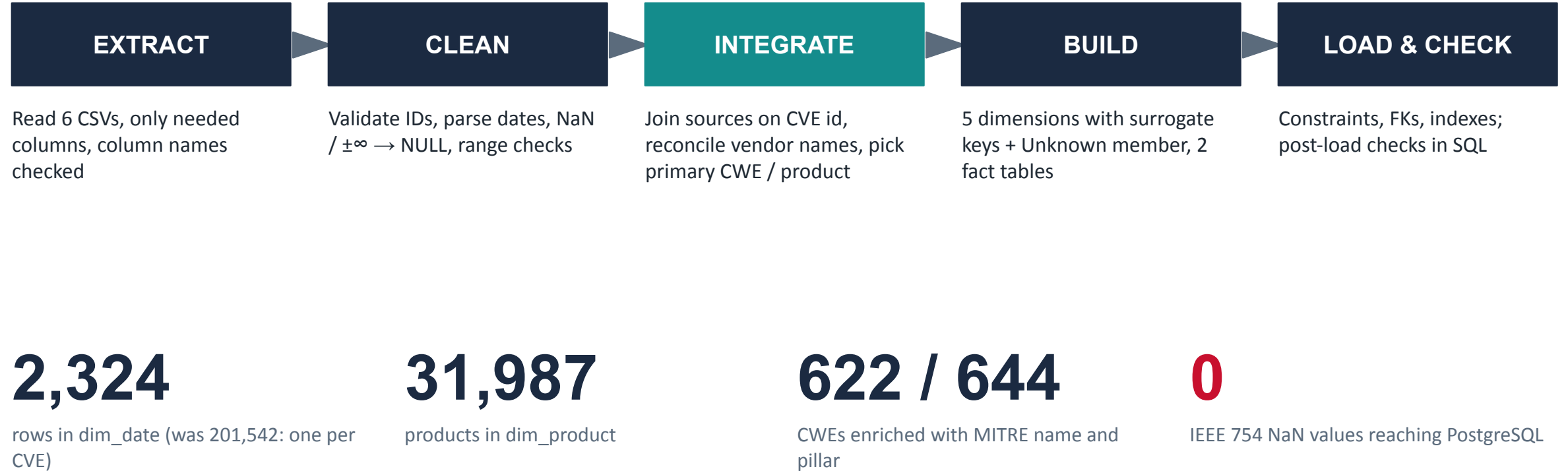
Data sources

Files from four providers, integrated by the ETL.

Source	Provider	File	Rows read	Used for
NVD CVE records	NIST	nvd_cve_records.csv	201,542	CVE, dates, CVSS, severity, attack vector
NVD weaknesses	NIST	nvd_cve_weaknesses.csv	260,862	CVE → CWE
NVD CPE matches	NIST	nvd_cve_cpe_matches.csv	1,550,739	CVE → vendor / product
EPSS scores	FIRST	first_epss_current.csv	329,934	probability of exploitation in 30 days
KEV catalog	CISA	cisa_kev_catalog.csv	1,585	confirmed exploitation, due date, ransomware
CWE catalog (view 1000)	MITRE	cwe_1000.csv	944	CWE names and CWE → pillar hierarchy

The Kaggle mirror ([afr1ste/nvd-cve-cvss-kev-vulnerabilities-2020-2026](#)) also ships a pre-joined table: it is used only to cross-check our joins, never as input.

ETL pipeline



ETL: what the transformations actually do

All counts from etl_report.json

Numeric cleaning

CVSS, EPSS and percentile are parsed with range checks (0–10, 0–1). Empty cells, 'NaN' and $\pm$ Infinity become NULL in pandas, so AVG and ORDER BY work without SQL workarounds.
8,974 CVEs without CVSS → NULL.

Time grain fixed

Timestamps are truncated to the calendar day.
dim_date is a full calendar (Jan 2020 – May 2026) with a smart key YYYYMMDD and month / quarter / year levels.
KEV dates use the same dimension.

Primary CWE and product

62,418 CVEs have several CWEs: the Primary one from NVD is kept. CPE strings are parsed (cpe:2.3:part:vendor:product); 445,848 non-vulnerable platform rows dropped; the most frequent vulnerable product is kept.

Entity resolution: CISA vendor → NVD vendor

258 vendor names in KEV: 149 identical to NVD, 62 remapped by majority vote over shared CVEs, 47 kept as in CISA.
d_link → dlink · check_point → checkpoint · android → google ·
barracuda_networks → barracuda

Hierarchy from MITRE

The ChildOf relations of CWE view 1000 are walked up to the root, giving the level cwe → pillar used for Roll-Up / Drill-Down (Q2).
NVD placeholders (NVD-CWE-noinfo, NVD-CWE-Other) are labelled as not classified.

ETL: validation

194,478

CVEs with an EPSS score (reference: 194,478)

1,039

CVEs of 2020–2026 found in KEV (reference: 1,039)

0

disagreements with the pre-joined KEV flag

3 / 3

post-load SQL checks passed

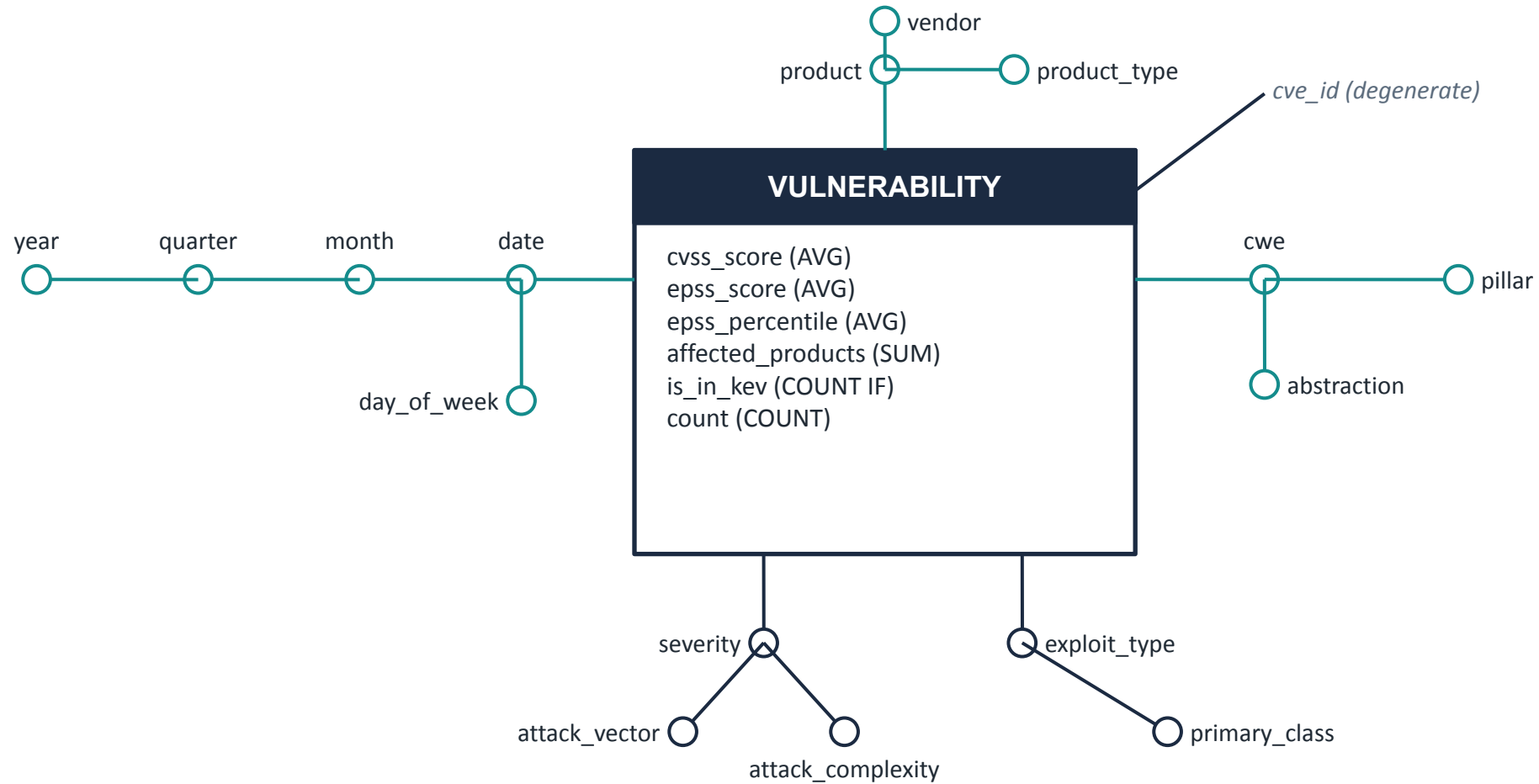
Post-load check (SQL, run by etl.py)	Result
dim_date rows = distinct dates + Unknown member	OK
fact_vulnerability.is_in_kev = KEV rows inside the NVD snapshot	OK
no NaN in any numeric measure	OK

Known limits, stated

40,398 CVEs (20%) have no CPE → product = Unknown.
546 KEV entries are CVEs older than 2020: kept in the KEV fact, flagged in_nvd_snapshot = false.
EPSS is a current score, not historical.

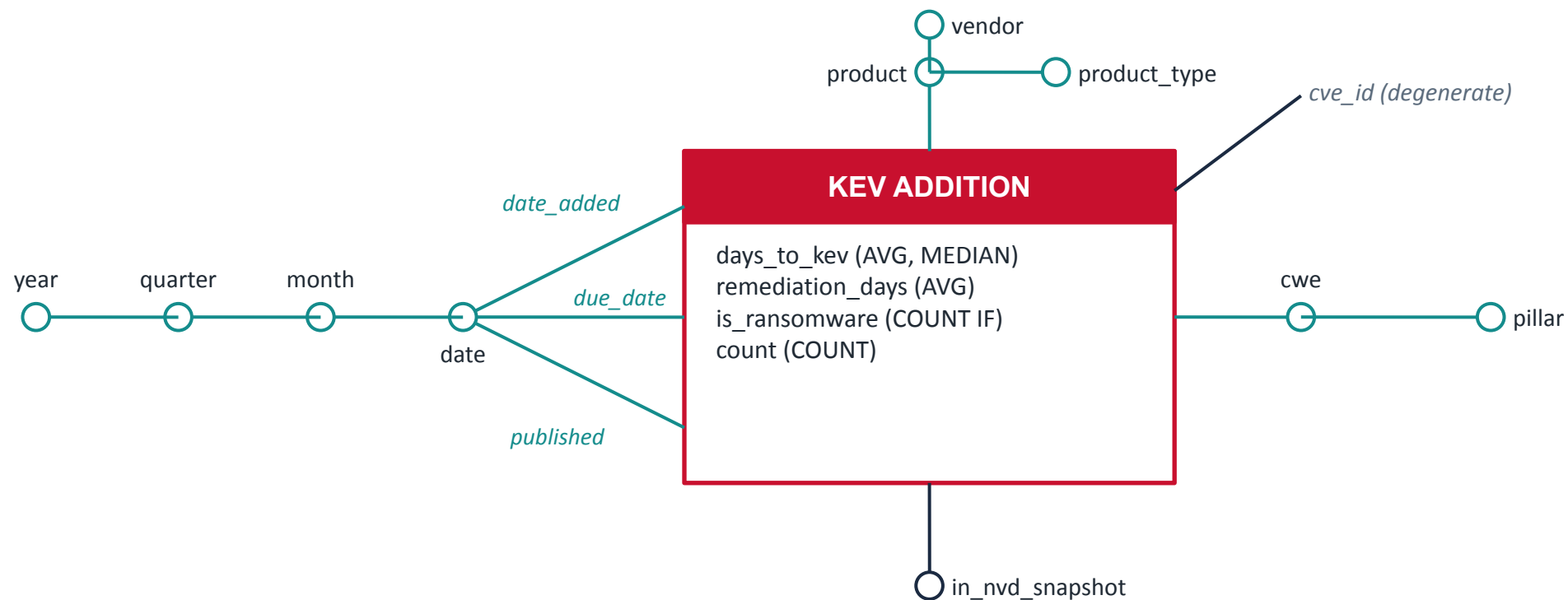
DFM 1: vulnerability disclosure

Grain: one CVE published in NVD



DFM 2: confirmed exploitation (KEV)

Grain: one CVE added to the CISA KEV catalog



Conformed dimensions date, product and cwe are identical in both facts: this is what makes the Drill-Across (Q6) possible.

Logical schema: fact constellation in PostgreSQL 16



Every dimension has an 'Unknown' member (key -1): facts never have NULL foreign keys and no row is lost in the joins.

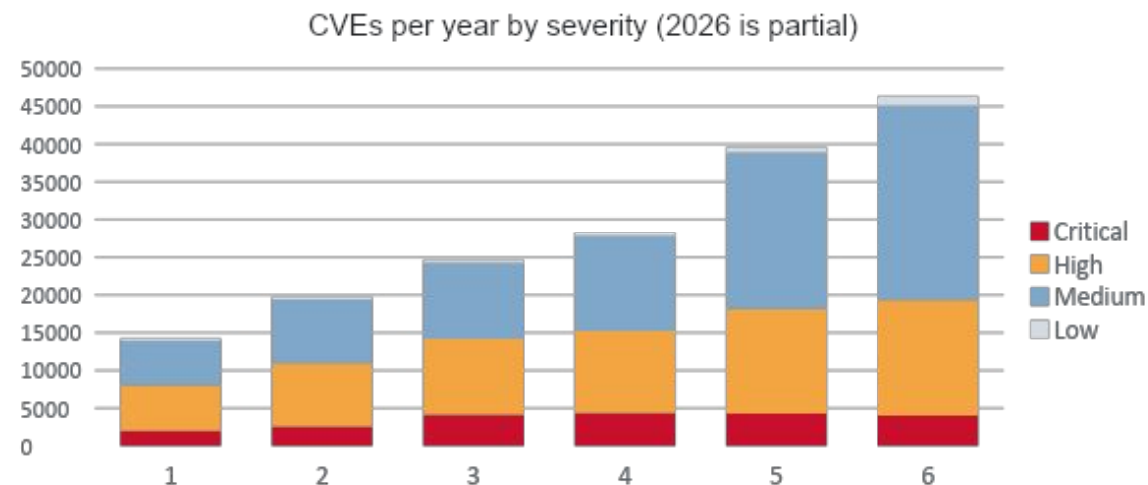
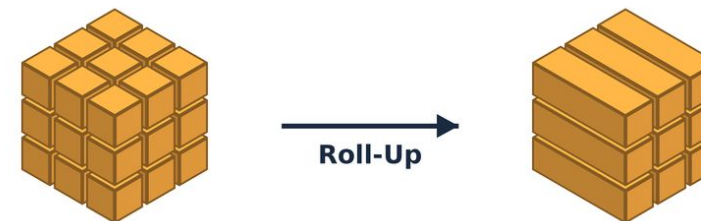
Q1 - Growth of disclosed vulnerabilities

ROLL-UP

"Number of CVEs per year and severity, rolled up from quarters to years."

```
SELECT d.year, d.quarter,
       COUNT(*) AS cve_count,
       COUNT(*) FILTER (WHERE s.severity_level='CRITICAL')
                           AS critical,
       ...
       ROUND(AVG(f.cvss_score),2) AS avg_cvss
FROM fact_vulnerability f
JOIN dim_date d
  ON d.date_key = f.published_date_key
JOIN dim_severity s
  ON s.severity_key = f.severity_key
WHERE d.date_key <> -1
GROUP BY ROLLUP (d.year, d.quarter)
ORDER BY d.year, d.quarter;

-- ROLLUP returns quarters, yearly
-- subtotals and the grand total
```



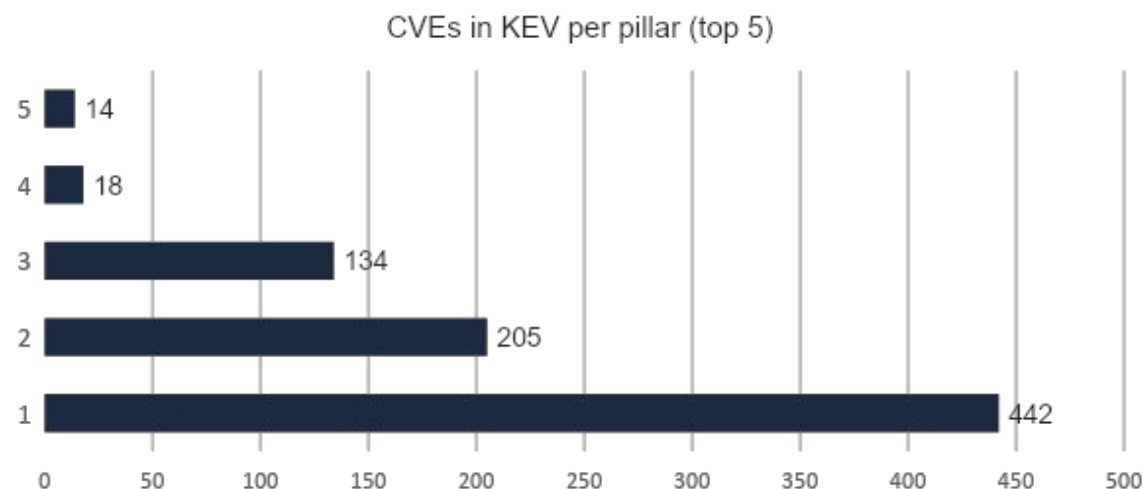
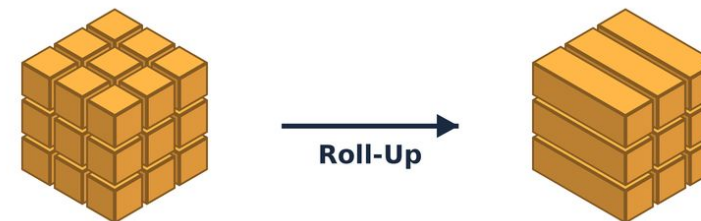
14,377 CVEs in 2020 → 49,549 in 2025 (×3.4). Critical ones grow less: from 14.0% to 8.1% of the total, and the average CVSS falls from 7.11 to 6.63.

Q2a · Weakness families that get exploited

ROLL-UP

“For each CWE pillar: how many CVEs, how many in KEV, what share is exploited?”

```
SELECT c.pillar_name,  
       COUNT(*) AS cve_count,  
       COUNT(*) FILTER (WHERE f.is_in_kev) AS in_kev,  
       ROUND(100.0 * COUNT(*) FILTER  
             (WHERE f.is_in_kev) / COUNT(*), 3)  
       AS exploitation_rate_pct,  
       ROUND(AVG(f.epss_score), 4) AS avg_epss  
FROM fact_vulnerability f  
JOIN dim_cwe c ON c.cwe_key = f.cwe_key  
WHERE c.pillar_id IS NOT NULL  
GROUP BY c.pillar_name -- cwe -> pillar  
ORDER BY in_kev DESC;
```



Memory-lifetime weaknesses and injection weaknesses have similar volumes (63k vs 57k CVEs), but memory ones are exploited twice as often: 0.70% vs 0.36%.

Q2b · Inside the most exploited family

DRILL-DOWN

“Open the pillar ‘Resource lifetime’ into its CWEs: most frequent vs most exploited.”

```
SELECT c.cwe_id, c.cwe_name,
       COUNT(*)                AS cve_count,
       COUNT(*) FILTER (WHERE f.is_in_kev) AS in_kev,
       RANK() OVER (ORDER BY COUNT(*) DESC)
                               AS rank_by_frequency,
       RANK() OVER (ORDER BY COUNT(*)
                    FILTER (WHERE f.is_in_kev) DESC)
                               AS rank_by_exploitation
FROM fact_vulnerability f
JOIN dim_cwe c ON c.cwe_key = f.cwe_key
WHERE c.pillar_name = <top pillar of Q2a>
GROUP BY c.cwe_id, c.cwe_name -- pillar -> cwe
ORDER BY in_kev DESC LIMIT 15;
```



CWE	Weakness	CVEs	In KEV	Rank freq.	Rank expl.
CWE-787	Out-of-bounds write	9,754	98	1	1
CWE-416	Use after free	4,511	65	4	2
CWE-502	Deserialization	2,013	46	8	4
CWE-843	Type confusion	469	27	29	5
CWE-125	Out-of-bounds read	5,372	12	2	9

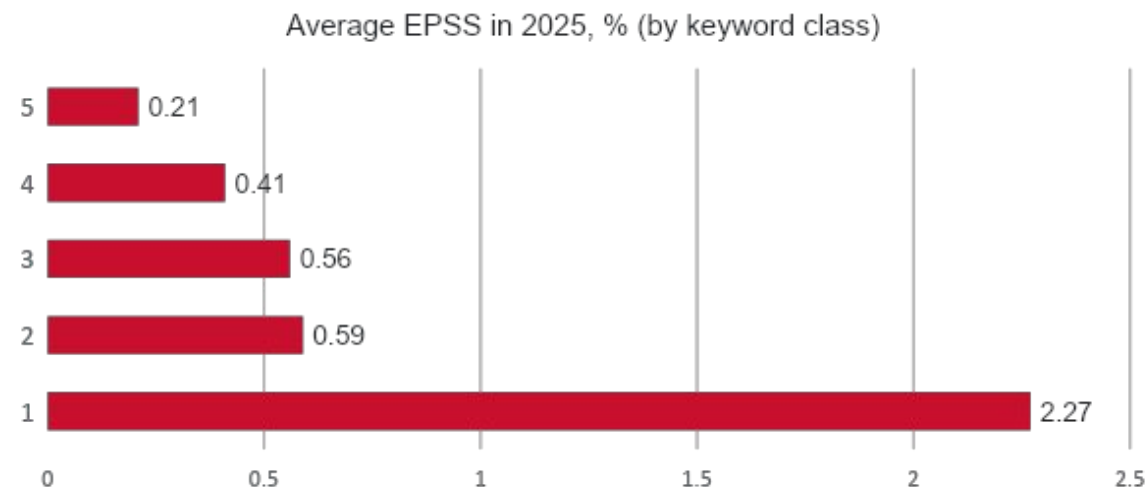
Frequency is a poor guide: type confusion is only #29 by volume but #5 by exploitation (5.8% of its CVEs are in KEV), while out-of-bounds read is #2 by volume and #9 by exploitation.

Q3 - Attack classes in 2025

SLICE

“Fix year = 2025: how do attack classes compare in severity and predicted exploitation?”

```
SELECT e.primary_class      AS exploit_class,
       COUNT(*)             AS cve_count,
       COUNT(*) FILTER (WHERE s.severity_level
                             IN ('CRITICAL','HIGH')) AS critical_or_high,
       ROUND(AVG(f.cvss_score), 2) AS avg_cvss,
       ROUND(AVG(f.epss_score), 4) AS avg_epss
FROM fact_vulnerability f
JOIN dim_date d
  ON d.date_key = f.published_date_key
JOIN dim_exploit_type e
  ON e.exploit_type_key = f.exploit_type_key
JOIN dim_severity s
  ON s.severity_key = f.severity_key
WHERE d.year = 2025      -- the slice
GROUP BY e.primary_class
ORDER BY avg_epss DESC;
```



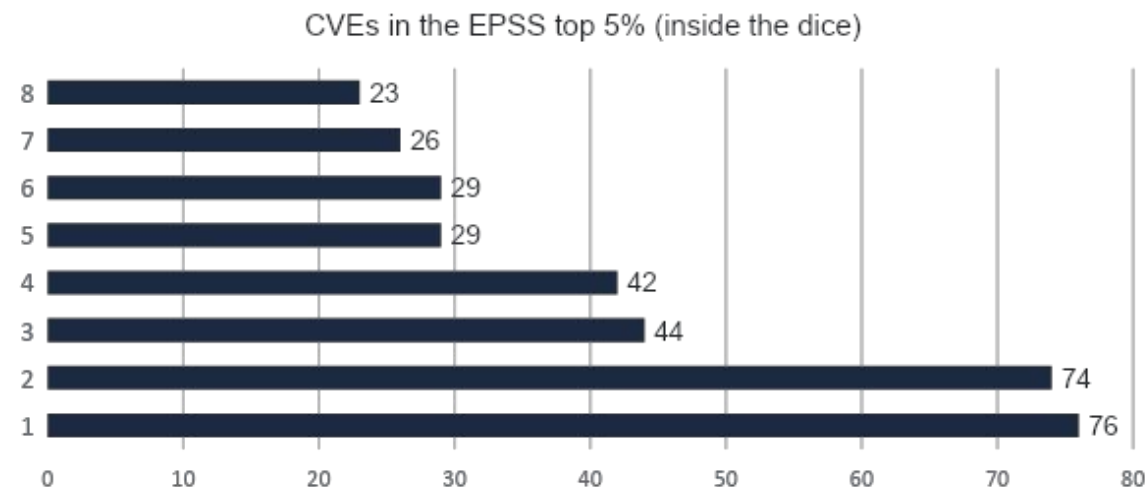
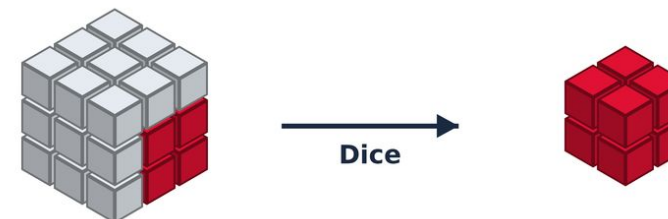
In 2025 remote code execution has an average EPSS 4× higher than any other class; XSS is the most harmless despite 7,147 CVEs.

Q4 - Riskiest vendors for severe remote flaws

DICE

“Severity CRITICAL/HIGH, network vector, 2022–2025, applications only: which vendors?”

```
SELECT p.vendor,
       COUNT(*) AS cve_count,
       COUNT(*) FILTER (WHERE f.is_in_kev) AS in_kev,
       COUNT(*) FILTER
         (WHERE f.epss_percentile >= 0.95) AS epss_top5pct
FROM fact_vulnerability f
JOIN dim_date d ON d.date_key = f.published_date_key
JOIN dim_severity s ON s.severity_key = f.severity_key
JOIN dim_product p ON p.product_key = f.product_key
WHERE s.severity_level IN ('CRITICAL', 'HIGH')
      AND s.attack_vector = 'NETWORK'
      AND d.year BETWEEN 2022 AND 2025
      AND p.product_type = 'application'
GROUP BY p.vendor
HAVING COUNT(*) >= 10
ORDER BY epss_top5pct DESC LIMIT 15;
```



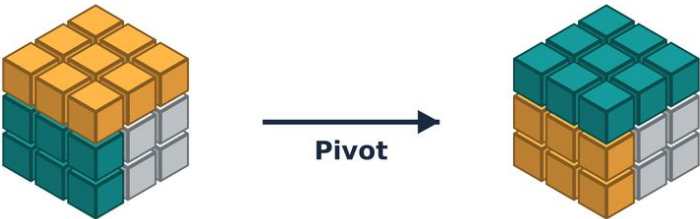
Ivanti has 76 of its 218 severe remote CVEs (35%) in the EPSS top 5%, against 13% for Apache and 7% for Microsoft: volume alone hides the riskiest vendor.

Q5 - Does CVSS agree with EPSS?

“Count CVEs by CVSS band (rows) and EPSS band rotated into columns.”

PIVOT

```
WITH banded AS (  
  SELECT CASE  
    WHEN cvss_score >= 9 THEN 'Critical'  
    WHEN cvss_score >= 7 THEN 'High'  
    WHEN cvss_score >= 4 THEN 'Medium'  
    ELSE 'Low' END AS cvss_band,  
    epss_score  
  FROM fact_vulnerability  
  WHERE cvss_score IS NOT NULL  
    AND epss_score IS NOT NULL)  
SELECT cvss_band,  
  COUNT(*) FILTER (WHERE epss_score < 0.01)  
    AS "EPSS <1%",  
  ...  
  -- 1-10%, 10-50%  
  COUNT(*) FILTER (WHERE epss_score >= 0.5)  
    AS "EPSS >=50%"  
FROM banded GROUP BY cvss_band;
```



CVSS \ EPSS	< 1%	1–10%	10–50%	≥ 50%
Critical	15,844	4,843	1,383	1,353
High	60,023	8,894	2,089	1,059
Medium	86,895	4,288	1,001	337
Low	4,419	114	24	1

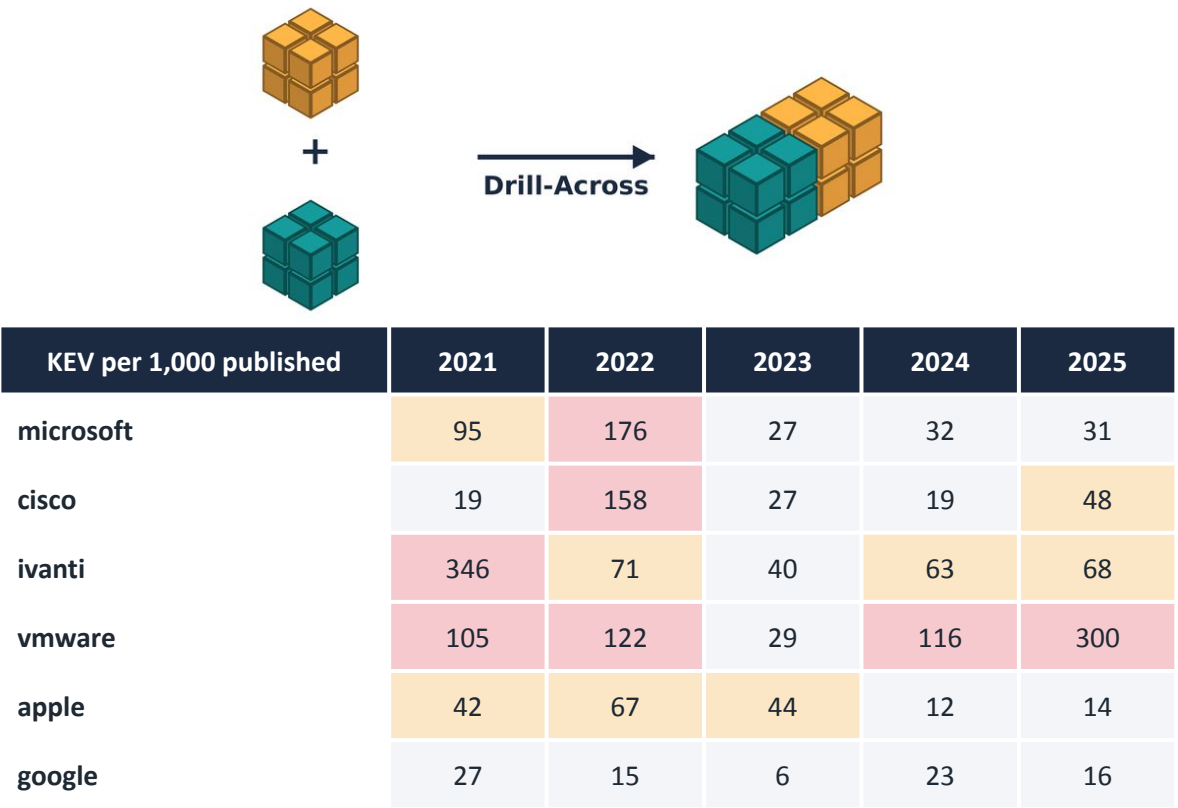
False alarms: 68% of CVSS-critical CVEs (15,844) have EPSS below 1%. Hidden dangers: 337 ‘medium’ CVEs have EPSS ≥ 50%. Patching by CVSS alone misprioritises.

Q6 - Disclosed vs exploited, per vendor and year

DRILL-ACROSS

“For each vendor and year: CVEs published (cube 1) and CVEs added to KEV (cube 2).”

```
WITH published AS (          -- cube 1
  SELECT p.vendor, d.year, COUNT(*) AS cve_published
  FROM fact_vulnerability f
  JOIN dim_product p USING (product_key)
  JOIN dim_date d ON d.date_key = f.published_date_key
  GROUP BY p.vendor, d.year),
exploited AS (              -- cube 2
  SELECT p.vendor, d.year, COUNT(*) AS cve_added_to_kev
  FROM fact_kev_addition k
  JOIN dim_product p USING (product_key)
  JOIN dim_date d ON d.date_key = k.date_added_key
  GROUP BY p.vendor, d.year)
SELECT vendor, year, cve_published, cve_added_to_kev,
  ROUND(1000.0 * cve_added_to_kev
    / NULLIF(cve_published, 0), 1) AS kev_per_1000
FROM published FULL OUTER JOIN exploited
  USING (vendor, year)      -- conformed dimensions
WHERE year BETWEEN 2021 AND 2025;
```



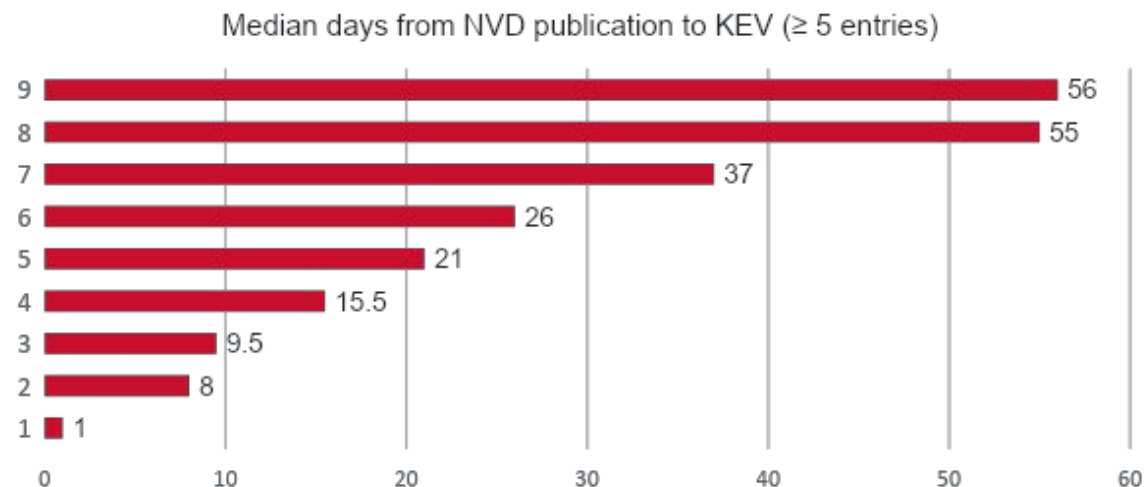
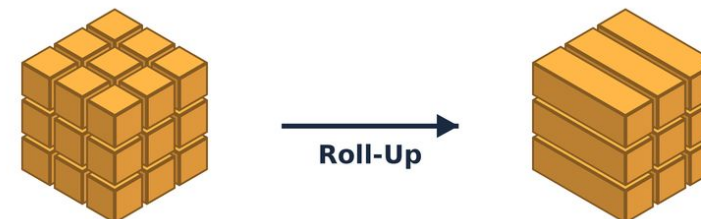
The 2021–22 peaks are the KEV back-fill (catalog created Nov 2021). From 2023 Microsoft stabilises around 30 per 1,000, while Ivanti (62–68) and VMware stay much higher.

Q7 - How fast are vulnerabilities exploited?

ROLL-UP

"On the KEV cube, roll up products to vendors: median days from publication to KEV."

```
SELECT p.vendor,
       COUNT(*) AS kev_entries,
       COUNT(*) FILTER
         (WHERE k.days_to_kev < 0) AS zero_day,
       PERCENTILE_CONT(0.5) WITHIN GROUP
         (ORDER BY k.days_to_kev)
         FILTER (WHERE k.days_to_kev >= 0)
         AS median_days_to_kev,
       ROUND(100.0 * COUNT(*) FILTER
         (WHERE k.is_ransomware) / COUNT(*), 1)
         AS ransomware_pct
FROM fact_kev_addition k
JOIN dim_product p ON p.product_key = k.product_key
WHERE k.in_nvd_snapshot
GROUP BY p.vendor -- product -> vendor
HAVING COUNT(*) >= 5
ORDER BY median_days_to_kev LIMIT 15;
```



Network appliances are exploited fastest: Fortinet in a median of 1 day, Palo Alto 8, Citrix 15.5. 61 CVEs entered KEV before NVD publication (zero-days), 16 each for Google and Apple.

Conclusions

Severity ≠ risk

68% of critical CVEs are almost never exploited (Q5); rare weaknesses like type confusion are exploited far above their volume (Q2b).

Vendors differ in kind

Ivanti and VMware are exploited at 2–10× Microsoft's rate per published CVE (Q6); network appliances are exploited within days (Q7).

Two cubes, one view

Conformed dimensions let the two business processes (disclosure and exploitation) be analysed together with a Drill-Across.

Limitations

- 20% of CVEs have no CPE, so their product is Unknown
- EPSS is a single current snapshot, not a history over time
- KEV lists confirmed exploitation only: absence from KEV does not mean “not exploited”
- Attack classes come from keyword matching on descriptions (heuristic)

Thank you for your attention

Code: `etl.py` · `queries.sql` · `export_results.py`